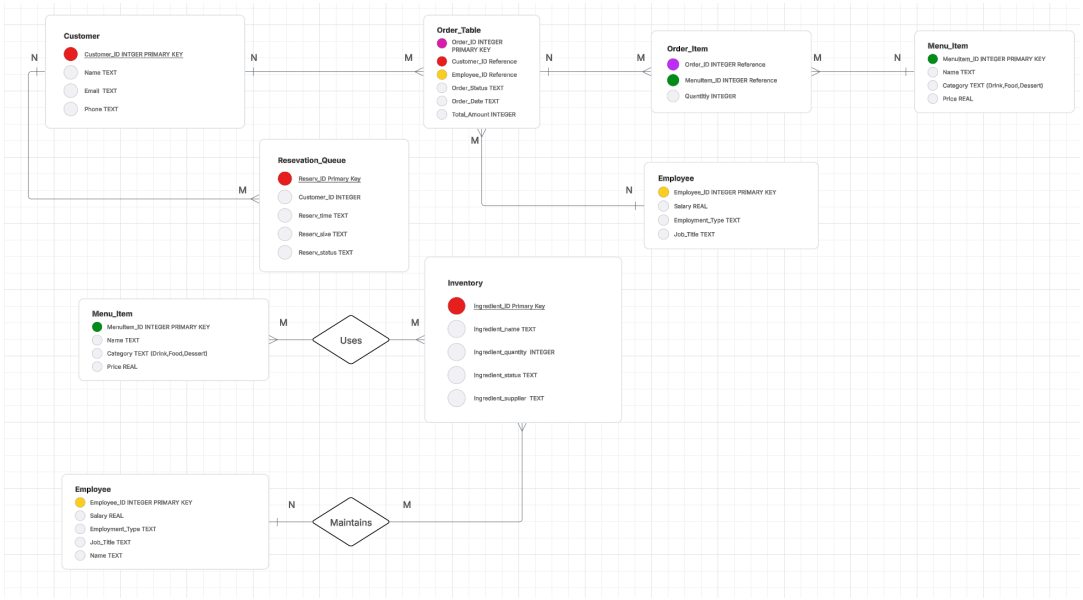
Project Report
Description of Enterprise

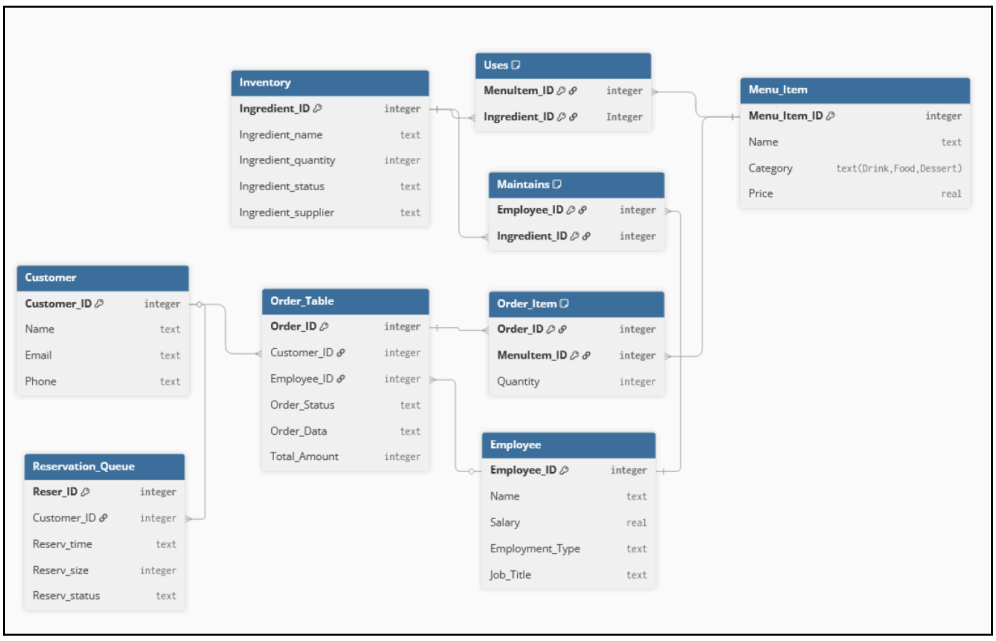
Our enterprise is an *aquarium-themed cafe*. As a business in the food service industry, it would need a way to keep track of pertinent metrics necessary for planning and continuous operation, including tracking what menu items it offers, what ingredients it has in stock, what employees it has hired, what orders have been made, and what customers have ordered or made a reservation. **A database is a convenient solution for being able to access and store this data in a consistent manner**, as otherwise the cafe would not be able to appropriately track owed wages or meet the demands of its customers– and could eventually lead to the failure of the business.

ER Diagram

[Link to Website where we made the ER Diagram]



Relational schema



Core Entity Relations

Relation Name	**Schema Notation**	**Description/Explanation**
Customer	Customer(**Customer_ID**,Name,Email,Phone)	The primary key is Customer_ID. Stores unique information about each **customer**.
Employee	Employee(**Employee_ID**,Name, Salary,Employment_Type,Job_Title)	The primary key is Employee_ID. Stores unique data for each **staff member**.
Menu_Item	Menu_Item(**Menu_Item_ID**,Name,Category,Price)	The primary key is Menu_Item_ID. Is a catalog for all the items available for order.
Inventory	Inventory(**Ingredient_ID**,Ingredient_name,Ingredient_quantity,Ingredient_status,Ingredient_supplier)	The primary key is Ingredient_ID. Tracks all the **raw ingredients** and their current stock levels.

Transactional Relations

Relation Name	**Schema Notation**	**Description**

Reservation_Queue	Reservation_Queue(**Reser_ID**, Customer_ID, Reserv_time, Reserv_size, Reserv_status)	Customer_ID is FK referencing Customer. Records details of a single **reservation**.
Order_Table	Order_Table(**Order_ID**,Customer_ID, Employee_ID,Order_Status, Order_Data, Total_Amount)	Customer_ID is FK to Customer. Employee_ID is FK to Employee. Records a single **order**.
Order_Item	Order_Item(**Order_ID**,**MenuItem_ID**,Quantity)	M:N relationships between Order_Table and Menu_Item. Links a single **Order** to multiple menu items.
Uses	Uses(**MenuItem_ID**,**Ingredient_ID**)	M:M relationships between Menu_Item and Inventory. Tracks recipe components.
Maintains	Maintains(**Employee_ID**,**Ingredient_ID**)	N:M relationships between Employee and Inventory. This tracks who is responsible for the last updated stock/status of a particular ingredient.

Test queries: A set of 5 test queries to test if the database is working as intendd
****Important****

1. **Please run "run_first.py" first to create the database, then proceed to run "run_test_queries.py" for the test queries**
2. **After running "run_test_queries.py", please delete "restaurant.db" and then rerun "run_first.py" again before using the CLI in "main.py". This is so that whatever is changed from the test queries to the database isn't reflected when using the CLI (for a fresh start).**

Test Query #1 Add Customer
Description: Inserting a new customer into the customer table
Outcome: Follow the format of (Customer_ID, Name, Email, Phone)

Before	After
(1, 'Alice Kim', 'alice@example.com', '555-1234') (2, 'Jamal Peterson', 'jamal@example.com', '555-5678') (3, 'Maria Lopez', 'maria@example.com', '555-9012')	(1, 'Alice Kim', 'alice@example.com', '555-1234') (2, 'Jamal Peterson', 'jamal@example.com', '555-5678') (3, 'Maria Lopez', 'maria@example.com', '555-9012') (4, 'Test Customer', 'testcustomer@example.com', '555-0101')

Test Query #2 View Reservations
Description: Select all current reservations in the system
Outcome: Follow the format of (Reserv_ID, Customer_ID, Reserv_time, Reserv_size, Reserv_status)

(1, 1, '2025-12-01 18:00', 2, 'Pending')

Test Query #3 Cancel Order
Description: Delete order items and order from the system given Order_ID = 3 (Maria Lopez)
(In the CLI, only allow to delete orders with Queue status to mimic real establishment)
Outcome: Follow the format of (Order_ID, Customer Name, Order_Status, Order_Date, Total_Amount)

Before	After
(1, 'Alice Kim', 'Alexander Farell', 'Finished', '2025-12-03 21:52:02', 19.3) (2, 'Jamal Peterson', 'Alexander Farell', 'Preparing', '2025-12-03 22:12:02', 11.65) (3, 'Maria Lopez', 'Alexander Farell', 'Queue', '2025-12-03 22:32:02', 15.4)	(1, 'Alice Kim', 'Alexander Farell', 'Finished', '2025-12-03 21:52:02', 19.3) (2, 'Jamal Peterson', 'Alexander Farell', 'Preparing', '2025-12-03 22:12:02', 11.65)

Test Query #4 Update Inventory
Description: Update an item given Ingredient_ID = 3 (Cocoa Powder) (Updated quantity)
Outcome: Follow the format of (Ingredient_ID, Ingredient_name, Ingredient_quantity, Ingredient_status, Ingredient_supplier)

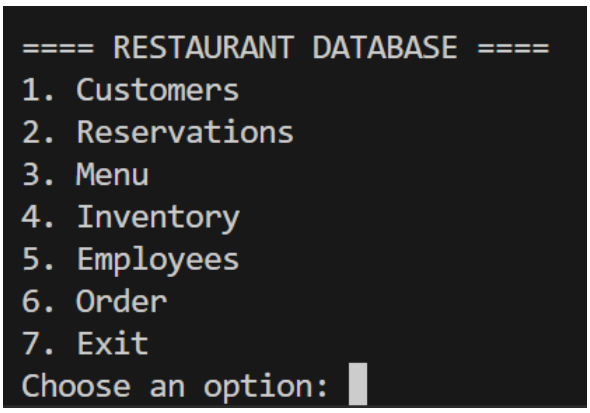
Before	After
(1, 'Milk', 40, 'In Stock', 'DairyCo') (2, 'Beef Patty', 20, 'Low Stock', 'MeatSupply') (3, 'Cocoa Powder', 15, 'In Stock', 'SweetFoods')	(1, 'Milk', 40, 'In Stock', 'DairyCo') (2, 'Beef Patty', 20, 'Low Stock', 'MeatSupply') (3, 'Cocoa Powder', 30, 'In Stock', 'SweetFoods')

Test Query #5 Delete Employee
Description: Delete an employee from the system given Employee_ID = 3 (Lucas Henderson)
Outcome: Follow the format of (Employee_ID, Name, Salary, Employment_Type, Job_Title)

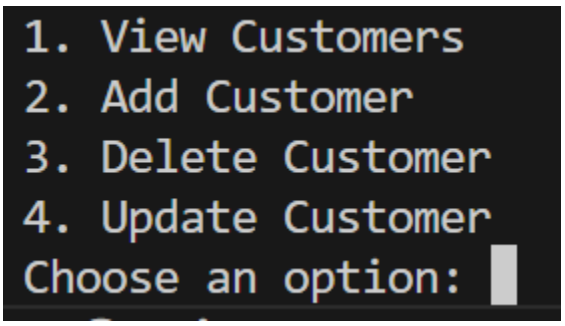
Before	After
(1, 'Alexander Farell', 18.5, 'Part-Time', 'Server') (2, 'Isabella Martinez', 22.0, 'Full-Time', 'Cook') (3, 'Lucas Henderson', 15.0, 'Part-Time', 'Host')	(1, 'Alexander Farell', 18.5, 'Part-Time', 'Server') (2, 'Isabella Martinez', 22.0, 'Full-Time', 'Cook')

Application user interface

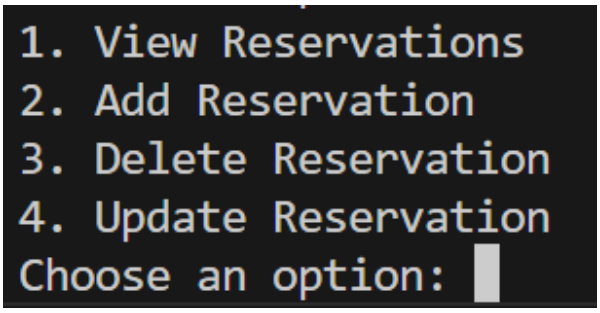
Main Interface

	This is the main interface that links to each specific interfaces depending on what the user chooses.

Customers

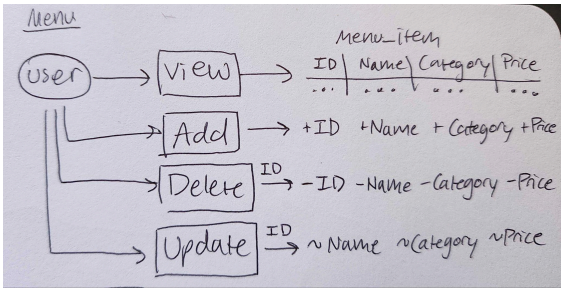
	The customer interface allows users to perform changes within the Customer table. The Customer table takes in credentials like name, email, and phone to add, delete, and update customer information. The user can also view all customers existing in the database. The expected user for this would be the **employees** of the cafe, most likely the **host**.

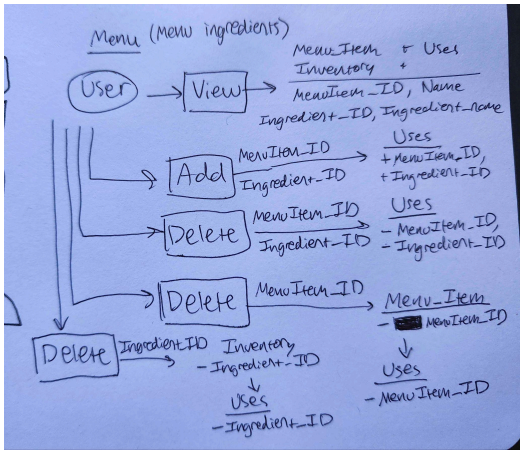
Reservations

	The reservations interface allows users to perform operations on the Reservation_Queue Table, allowing you to view, add, delete, and update items in the table. The expected user for this would be **employees** of the cafe.

Menu

Menu Item Operations:	Menu Item Ingredient Operations:





CLI:

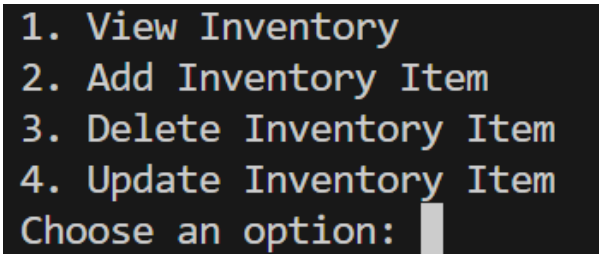
```
1. View Menu Items
2. Add Menu Item
3. Delete Menu Item
4. Update Menu Item
5. View Menu Item Ingredients
6. Add Menu Item Ingredient
7. Delete Menu Item Ingredient
Choose an option:
```

This interface allows you to edit the menu by performing operations on the Menu_Item and Uses table. Menu items can be viewed, added, deleted, or updated.

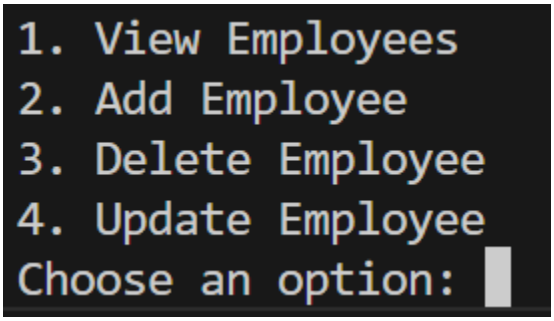
The Uses table tracks menu item ingredients - which ingredients are required for which menu items. Menu item and ingredient relationships can be viewed, added, or deleted. The interface uses the Menu_Item and Inventory tables in conjunction with the Uses table to display both the item IDs and their names when a user views menu item ingredients. When a menu item or ingredient is deleted, all of the relevant menu item or ingredient relationships in the Uses table are deleted.

The expected user would be the **cafe owner** and **employees**.

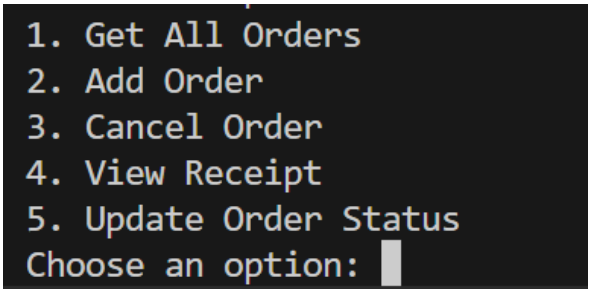
Inventory

	This interface allows you to view, add, delete, and update the ingredients that the cafe has available for its menu items. This is tracked in the Inventory table and referenced by the Uses table (for tracking which menu items use which ingredients). Deleting an inventory item deletes all of its relevant relationships in the Uses table. For each ingredient, the name, quantity, status, and supplier are tracked and modifiable. The expected user would be **employees**.

Employee

	This interface allows you to view, add, delete & update the status of the employees of the cafe. This is tracked in the Employee table and referenced by the Order_Table table (tracking which employee is assigned to handle a particular order). For each employee, their name, salary, employment type and job title are tracked. Their employment type refers to whether they are part-time or full time. Their job title is referenced when placing orders, as only Servers can fulfill orders. The expected user would be the **cafe owner**.

Orders

	1. Picking this will display a list of all orders in the system 2. Picking this brings up a list of customers to choose from, then a list of applicable employees (Server title only) to fulfill the order, and finally a list of menu items and its quantity to complete the order. (When typing 'done', please note it is case-sensitive) 3. Picking this will bring up a list of orders with status 'Queue" to cancel (Can only cancel orders that are

	queued. 4. Picking this will bring up a list of orders with status 'Finished' to print a receipt for (Can only have a receipt for orders that are finished) 5. Picking this will bring up a list of orders with status that can be updated (Finished orders cannot go backward). Status changes are also forward only, meaning 'Preparing' cannot go back to 'Queue'. (When changing status, please note it is case-sensitive) This interface would be typically used by the **employees**, most likely **servers** when they are inputting orders for customers.

Participation details

Ella
- Drafted the ER Diagram with Nguyen and Arabelle
- Implemented the Relationship Schema into SQL.
- Added setup for database, schema, and seed data for "run_first.py".
- Implemented Reservation interface (and its report details),

Nguyen
- Implemented order CRUD and CLI (and its report details).
- Made the 5 test queries in "run_test_queries.py" (and its report details).
- Implemented Customer interface (and its report details),

Kashvi
- Implemented employee CRUD and CLI
- Contributed to the Relationship Schema.

Arabelle
- Implemented menu CRUD and CLI.
- Implemented the Relationship Schema Diagram.
- Created enterprise and menu interface descriptions.

Entire Team
- Active Communication in Team Chat
- Github collaboration and updates
- Updating Final Report